

AgenticSciML: collaborative multi-agent systems for emergent discovery in scientific machine learning

Received: 30 November 2025

Accepted: 3 April 2026

Cite this article as: Jiang, Q., Karniadakis, G. AgenticSciML: collaborative multi-agent systems for emergent discovery in scientific machine learning. *npj Artif. Intell.* (2026). <https://doi.org/10.1038/s44387-026-00102-5>

Qile Jiang & George Karniadakis

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

AGENTICSciML: COLLABORATIVE MULTI-AGENT SYSTEMS FOR EMERGENT DISCOVERY IN SCIENTIFIC MACHINE LEARNING

Qile Jiang

Division of Applied Mathematics
Brown University
Providence, RI, USA
qile_jiang@brown.edu

George Karniadakis *

Division of Applied Mathematics
Brown University
Providence, RI, USA
george_karniadakis@brown.edu

ABSTRACT

Scientific Machine Learning (SciML) integrates data-driven inference with physical modeling to solve complex problems in science and engineering. However, the design of SciML architectures, loss formulations, and training strategies remains an expert-driven research process, requiring extensive experimentation and problem-specific insights. Here we introduce AgenticSciML, a collaborative multi-agent system in which over 10 specialized AI agents collaborate to propose, critique, and refine SciML solutions through structured reasoning and iterative evolution. The framework integrates structured debate, retrieval-augmented method memory, and ensemble-guided evolutionary search, enabling the agents to generate and assess new hypotheses about architectures and optimization procedures. Across physics-informed learning and operator learning tasks, the framework discovers solution methods that outperform single-agent and human-designed baselines by up to four orders of magnitude in error reduction. The agents produce novel strategies—including adaptive mixture-of-expert architectures, decomposition-based PINNs, and physics-informed operator learning models—that do not appear explicitly in the curated knowledge base. These results show that collaborative reasoning among AI agents can yield emergent methodological innovation, suggesting a path toward scalable, transparent, and autonomous discovery in scientific computing.

Introduction

Scientific Machine Learning (SciML) integrates data-driven learning with numerical simulation and physical modeling to solve problems in scientific computing that are analytically intractable or computationally expensive. Recent developments such as physics-informed neural networks (PINNs)¹, neural operators^{2,3}, and domain-decomposition methods^{4,5} have enabled progress across fluid mechanics, inverse problems, materials modeling, biophysics, and geoscience. However, the construction of effective SciML models still requires substantial manual effort: practitioners must select architectures, enforce physical constraints, tune loss weightings, design sampling curricula, and determine proper optimization strategies. This iterative search is labor-intensive and problem-specific, forming a bottleneck for scalable scientific computing.

Recent LLM-based agent systems have motivated interest in automating portions of the SciML workflow^{6,7}. Notable examples include the *Virtual Lab*⁸, in which domain-specialized agents collaborate to plan experiments and interpret results, and ChemCrow, an agent-based framework for molecular synthesis planning⁹. These approaches demonstrate the utility of role-specialization and iterative refinement, but they focus primarily on managing experimental or model-selection workflows. More broadly, agentic systems have been applied to open-ended scientific discovery^{10–13}, mathematical exploration and algorithm synthesis^{14–16}, scientific equation discovery¹⁷, automated data science and code exploration^{18,19}, numerical computation²⁰, symbolic regression^{21,22}, and expert-level scientific software development^{23,24}, while verification and reliability have emerged as key challenges in such pipelines^{25,26}. In notable frameworks such as AlphaEvolve²⁷ and ShinkaEvolve²⁸, evolutionary and program-synthesis approaches are employed to iteratively improve solutions, leading to potentially new scientific and practical discoveries. In parallel, for machine

*Corresponding author: george_karniadakis@brown.edu

learning research, AutoML systems and neural architecture search (NAS) frameworks^{29;30} optimize hyperparameters or network topologies, with recent work extending this to agent-driven architecture discovery³¹. Within SciML specifically, recent LLM-based systems have been proposed for automated PINN construction and PDE surrogate modeling^{32;33}. However, these systems do not generally produce *new SciML modeling strategies*, such as novel PDE-constrained architectures, problem-specific loss formulations, or adaptive decomposition schemes. Instead, they typically search within a predetermined hypothesis space or rely on a fixed modeling paradigm.

In contrast, scientific computing often requires reasoning about model structure, discretization, and physics-informed inductive bias. The space of possible SciML solution strategies is therefore combinatorial and structured, involving architectural motifs, physics constraints, numerical solvers, and multi-objective optimization. A system capable of exploring this space must not only tune parameters but also *compose and critique modeling ideas*.

To address this need, drawing inspiration from recent agentic systems for iterative scientific software refinement^{23;24}, we introduce **AgenticSciML**, a collaborative multi-agent framework for the *emergent discovery of SciML modeling strategies*. The framework coordinates multiple specialized agents—including proposers, critics, engineers, retrievers, and evaluators—that iteratively generate, analyze, and refine candidate SciML solutions. A persistent knowledge base stores prior solutions, ablation outcomes, error analyses, and textual reports. Retrieval agents surface relevant methodological precedents, allowing new strategies to build upon earlier ones. A structured debate process, grounded in evidence that multi-agent debate and ensemble voting improve decision quality³⁴, encourages agents to justify, challenge, and revise modeling decisions, while an ensemble-guided evolutionary search balances exploitation of promising directions with exploration of alternative approaches.

Across benchmarks in PDE-constrained learning, operator learning, and inverse problems, AgenticSciML produces solution strategies that outperform single-agent LLM systems, sometimes by 1000 $\times$, automated PINN frameworks³⁵, and NAS/AutoML baselines. Moreover, the system discovers SciML strategies that do not appear in the knowledge base or in standard formulations, including (1) adaptive domain-decomposed PINNs for multi-scale PDEs, (2) physics-informed operator learning architectures with constraint-conditioned branches, and (3) dynamically weighted loss schedules derived from residual-flow structure. These results indicate that structured multi-agent collaboration can enable *emergent innovation in SciML model construction*, beyond what can be achieved through direct prompting or search alone.

The remainder of this paper is organized as follows. In the Results section we detail experiment setups and findings across benchmark SciML problems. In the Discussion section we summarize our contributions and discuss limitations and future research directions. In the Methods section we describe the multi-agent architecture, including agent roles, knowledge retrieval mechanisms, and evolutionary search strategies.

Results

To demonstrate the capabilities of our multi-agent SciML system, we evaluate its performance on a challenging set of benchmark problems on physics-informed machine learning and operator learning. The problems include:

1. **Discontinuous Function Approximation (Supplementary Section S1.1):** Learning a piecewise oscillatory function with discontinuities from data using neural networks.
2. **Solving Poisson Equation on L-shaped Domain with PINN (Supplementary Section S1.2):** Using Physics-Informed Neural Networks (PINNs) to solve the Poisson equation on an L-shaped domain.
3. **Solving Burger’s Equation using PINN (Supplementary Section S1.3):** Using PINNs to solve the time-dependent nonlinear Burger’s equation with periodic boundary conditions and a challenging initial condition.
4. **Antiderivative Operator Learning (Supplementary Section S1.4):** Learning a neural operator that maps input functions to their antiderivatives, generalizing to unseen functions.
5. **Multiple-input Operator Learning for Reaction-Diffusion Equation (Supplementary Section S1.5):** Learning a neural operator that maps multiple input functions (diffusion coefficient and source term) to the solution of a reaction-diffusion equation over time.
6. **Reconstruction of 2D Cylinder Wake Vorticity Field from Sparse and Noisy Observations (Supplementary Section S1.6):** Using neural networks to reconstruct the vorticity field behind a cylinder from only 4 sensors and noisy measurements.

These problems are designed to encourage original thinking from the agents and be solved with novel SciML techniques. Detailed setups and results for each problem are provided in Supplementary Materials. All problems use the same knowledge base and similar agent parameters. Below, we summarize key findings across all experiments.

Multi-Agent Collaboration Outperforms Single-Agent Baselines

For all problems, a single root engineer agent generates the root solution as a baseline to improve upon. With today's state-of-the-art LLMs, single-agent systems can already produce reasonably competent solutions for many SciML problems. However, the multi-agent system discovers solutions that significantly outperform the single-agent baseline across all benchmark problems, with improvement factors ranging from $10\times$ to over $11,000\times$. Figure 1 summarizes the performance improvements. The improvement factor is measured as the ratio of the root solution score to the champion solution score after multi-agent evolution.

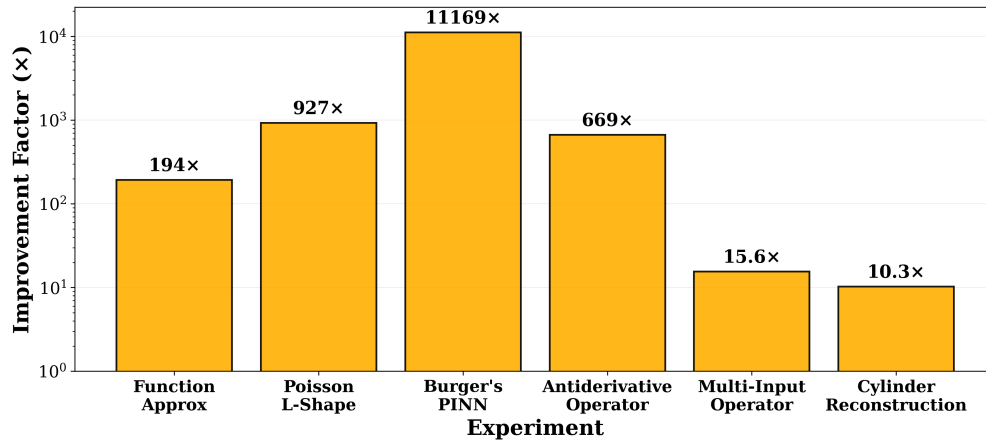


Figure 1

These results critically demonstrate that the intelligence of the multi-agent system emerges from collaborative reasoning, knowledge integration, and evolutionary tree search, rather than solely relying on the capabilities of individual LLMs.

Novel Solution Strategies Emerge Through Evolution

We define a solution strategy as an *emergent discovery* if it is not present in any KB entry but is instead synthesized by the agents through reasoning about retrieved techniques, problem structure, and prior experimental results. The KB provides methodological inspiration; emergence occurs when agents adapt, combine, or derive new approaches that extend beyond what any single KB entry explicitly describes.

We observe that the multi-agent system discovers novel solution strategies that differ significantly from the initial root solution. Even though knowledge base entries are retrieved and considered, the final solutions are not direct copies of existing methods. Instead, agents creatively adapt and combine techniques to devise new approaches tailored to each problem. Some novel strategies for each problem include:

1. **Discontinuous Function Approximation (Supplementary Section S1.1):** A Mixture-of-Experts architecture with specialized experts for different branches of the piecewise function, controlled by a learnable gating mechanism. The trainable gating parameter is inspired by the trainable parameter in adaptive activation functions³⁶.
2. **Poisson Equation on L-shaped Domain (Supplementary Section S1.2):** Decomposition of the target solution into a known particular solution plus a component learned by a PINN. The solution also uses importance sampling of collocation points to bias towards the corner's singularity.
3. **Burger's Equation (Supplementary Section S1.3):** Dividing training into 3 steps: pre-train only on the initial and boundary condition, then train on the gradient-enhanced PDE residual³⁷ with self-adaptive weights³⁸, and finally fine-tune with residual-based adaptive refinement of collocation points using double-precision L-BFGS.
4. **Antiderivative Operator Learning (Supplementary Section S1.4):** A DeepONet with a linear, bias-free branch net to enforce linearity of the antiderivative operator.
5. **Multiple-input Operator Learning (Supplementary Section S1.5):** 2D FNO trained with a derivative-enhanced loss, inspired by³⁹.
6. **Cylinder Wake Reconstruction (Supplementary Section S1.6):** Modified U-FNO architecture³⁹ with CNO-inspired⁴⁰ bandlimit-preserving filters in the decoder to mitigate aliasing during upsampling.

In particular, solutions to Problems 2 and 4 are mainly based on analysis of the underlying mathematical properties of the problem. Solutions to Problems 1, 5, and 6 come from combining and refining existing methods from the literature. Table 1 maps all KB entries retrieved during each experiment to the emergent strategy of the champion solution and compares what was retrieved versus what was synthesized.

Table 1: KB entries retrieved across all evolutionary iterations (deduplicated by reference) and the emergent discovery of the champion solution for each experiment. Strategies are emergent if not present in any retrieved KB entry. See Supplementary Section S1 for full details.

Problem	KB Entries Retrieved	Emergent Discovery (Champion)
Func. Approx.	Adaptive Act. ³⁶ ; Local Adaptive Act. ⁴¹ ; MoE-PINN ⁴² ; Self-Adaptive PINN ³⁸ ; gPINN ³⁷	MoE with learnable sigmoid gating; bounded sharpness k via $\sigma(k_{\text{raw}})$
Poisson	gPINN ³⁷ ; hp-VPINN (L-shaped) ⁴³ ; Self-Adaptive PINN ³⁸	Decomposition $u = u_{nn} + u_p$; power-law importance sampling at the corner
Burger's	gPINN ³⁷ ; Self-Adaptive PINN ³⁸ ; MoE-PINN ⁴² ; Local Adaptive Act. ⁴¹	3-phase training: BC/IC pretraining $\rightarrow$ gPINN + adaptive weights $\rightarrow$ RAR + L-BFGS
Antiderivative	DeepONet ³ ; PI-DeepONet ⁴⁴ ; Adaptive Act. ³⁶ ; Self-Adaptive PINN ³⁸ ; gPINN ³⁷	Linear bias-free branch net enforcing operator linearity
Multi-input Op.	U-FNO ³⁹ ; gPINN ³⁷ ; CNO ⁴⁰	1D inputs expanded into 2D spatiotemporal grids; hard BC/IC enforcement on FNO output
Cylinder Wake	U-FNO ³⁹ ; CNO ⁴⁰	CNO-inspired bandlimit-preserving Gaussian filter in U-FNO decoder upsampling

Ensemble Voting Analysis

The solution tree mutation is guided by an ensemble of selector agents. Throughout this work, Gemini-2.5-pro, GPT-5-mini, and Grok-4-fast are used as the selector ensemble due to their diverse perspectives. In all experiments, each selector nominates three candidate solutions for mutation. Figure 2 shows the ensemble's voting agreement for the top three nominations across all experiments.

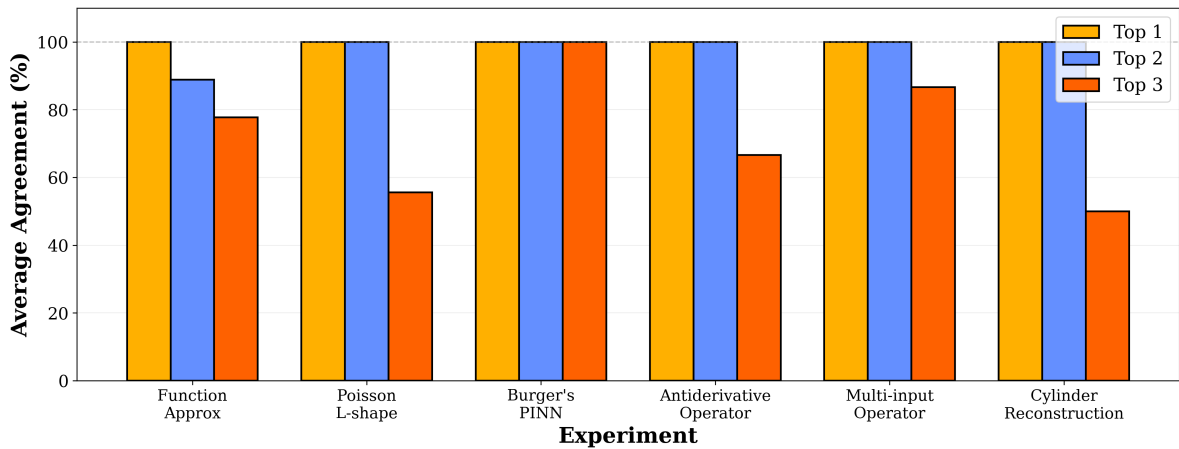


Figure 2

The selectors consistently agree on the first choice and mostly on the second, as these typically correspond to the best-performing solutions favored for exploitation. In contrast, the third selection shows a healthy disagreement between agents. This is because the third selection is usually an exploratory choice, where agents diverge on which solution has the potential for improvement. The voting mechanism thus introduces stochasticity and diversity into the evolutionary tree search.

Agent Contribution Analysis

Figure 3 summarizes the contributions of key planning agents by word count. The contributions from engineers and debuggers are not included. We observe that the proposer agent consistently contributes the most words, since it is instructed to think out loud and write down all intermediate reasoning steps. The human user contributes less than 0.3% of the total text, showing the system’s ability to autonomously handle problems with minimal human input.

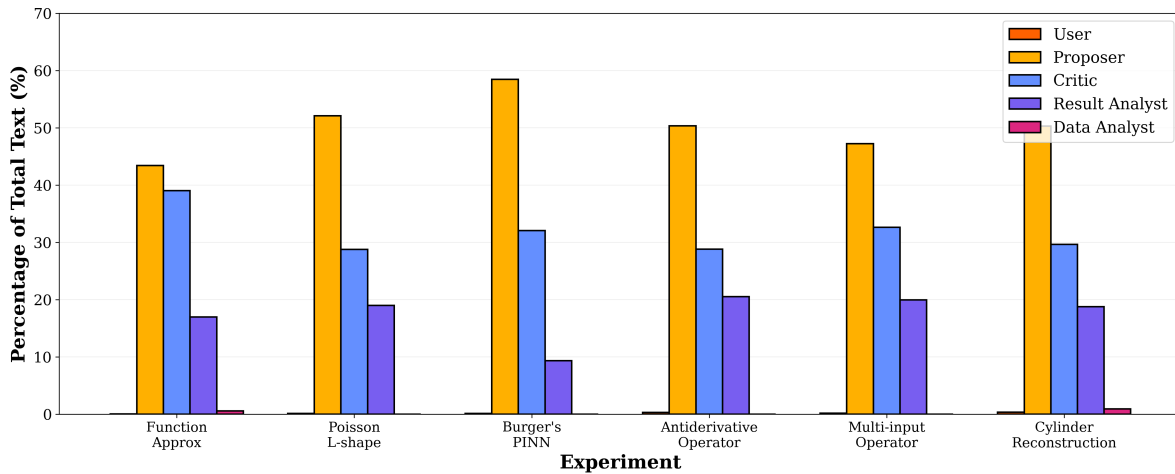


Figure 3

Token and Cost Analysis

The scalability and efficiency of the proposed framework are demonstrated through the following analysis of the associated computational costs and wall-clock execution time.

Total LLM API cost of one end-to-end experiment ranges from \$2.07 (Burgers’ PINN, 3 evolution iterations) to \$11.30 (function approximation, 6 evolution iterations), showing that the multi-agent evolutionary framework is practical at roughly single-digit dollar cost per experiment and readily scalable to larger-scale research projects. Across all six problems, the proposer agent dominates LLM expenditure in most runs (36–50% of total LLM cost in four of six experiments), since it generates and debates multiple candidate solutions per iteration. The result analyst also incurs a large token cost, owing to multimodal inputs that include training loss curves and plots. In most experiments, GPU training time substantially exceeds LLM API calls’ time (e.g., 5.6 h GPU vs. 1.7 h LLM for the Poisson problem; 10.7 h vs. 2.1 h for multi-input operator learning), showing that the execution time bottleneck is solution-specific model training rather than multi-agent coordination.

All experiments were run on NVIDIA A6000 GPUs. Detailed per-agent breakdowns are tabulated in the Supplementary section.

Knowledge Base Ablation

We evaluate KB sensitivity on one controlled benchmark: discontinuous function approximation (Supplementary Section S1.1). The objective is to test whether performance depends on retrieving relevant KB entries, and how the multi-agent reasoning trajectory changes when KB support is removed or made irrelevant.

We ran three settings with all non-KB factors fixed:

- Full KB: standard retrieval from the 70-entry KB.
- No KB: retriever disabled (no KB context provided to proposer/critic).
- Random KB: retriever returns a random KB entry each mutation.

All three runs use the same root solution (`solution_0`), the same evaluation contract (`evaluate.py`, `guidelines.md`), the same agent configuration (Supplementary Table S1), and the same evolutionary parameters (6 iterations, mutation batch size 4), with the same training and testing sets. All three settings start from the same root solution. Relative to Full KB, No-KB achieves $2.3\times$ higher error and Random-KB $20.7\times$ higher error.

Table 2: KB ablation results for discontinuous function approximation (lower is better).

Setting	Best solution	Best MSE
Full KB	<code>solution_0100</code>	1.4631×10^{-3}
No KB	<code>solution_00101</code>	3.4208×10^{-3}
Random KB	<code>solution_00002</code>	3.0315×10^{-2}

Table 3: Round-3 proposer trajectory comparison across KB settings (function approximation only).

Setting	Best lineage	Observed method trajectory
Full KB	0 $\rightarrow$ 01 $\rightarrow$ 010 $\rightarrow$ 0100	Trajectory converges to a stable MoE strategy: feature-augmented adaptive activations (01) $\rightarrow$ explicit two-expert gating (010) $\rightarrow$ bounded trainable gate sharpness and interface-focused weighting (0100). Retrieved entries are directly relevant, and later rounds refine rather than replace the core mechanism.
No KB	0 $\rightarrow$ 00 $\rightarrow$ 001 $\rightarrow$ 0010 $\rightarrow$ 00101	Agents still discover domain-decomposition and gating ideas from internal reasoning, but with more trial-and-error pivots: soft temperature gate $\rightarrow$ hard detached gate $\rightarrow$ Gumbel-softmax routing $\rightarrow$ rollback to simpler asymmetric experts with a stable sigmoid gate and explicit jump term. Improvement is substantial, but less efficient and less stable than Full KB.
Random KB	0 $\rightarrow$ 00 $\rightarrow$ 000 $\rightarrow$ 0000 $\rightarrow$ 00002	Retrieved papers are frequently off-task (e.g., PINN/DeepONet/GNOT/ev-NSFnet entries from different PDE/operator regimes). The trajectory is less coherent: continuity-heavy and auxiliary-complex detours are introduced, then partially removed. Final recovery comes from simplifying back to a cleaner MoE-like formulation, but quality remains much worse than Full KB.

These results indicate that success is not purely dependent on retrieving one exact “right” paper: even without KB, the system still iteratively improves candidate solutions from 2.8288×10^{-1} to 3.4208×10^{-3} through multi-agent reasoning alone, although the final result is less optimal than Full KB. At the same time, performance is strongly correlated with KB relevance. Relevant retrieval helps the proposal-critic loop converge to a coherent method and best score, while random/irrelevant retrieval degrades outcome quality and can misguide agent reasoning toward less relevant design choices.

Discussion

We introduced **AgenticSciML**, a collaborative multi-agent framework for constructing scientific machine learning (SciML) models through structured proposal, critique, refinement, and evaluation. In contrast to automated hyperparameter tuning, one-shot prompting, or neural architecture search pipelines, AgenticSciML coordinates distinct reasoning roles among agents and maintains a persistent methodological memory, enabling the system to explore the space of modeling strategies rather than merely parameter configurations. The resulting designs are not limited to variations of standard templates; instead, the system produces new forms of SciML architectures and training procedures that reflect nontrivial modeling decisions, such as adaptive domain decomposition in PINNs, constraint-conditioned branches in operator networks, or dynamically modulated loss schedules based on residual structure. These outcomes are characteristic of expert manual development in scientific computing and indicate that structured agentic collaboration can support emergent model-form innovation.

The approach shares conceptual motivation with recent multi-agent scientific planning frameworks such as the Virtual Lab⁸ and ChemCrow, an agent-based framework for molecular synthesis planning⁹, as well as with neural architecture search and evolutionary optimization methods^{29,30}. However, those systems typically optimize within fixed hypothesis classes or focus on experimental decision-making, whereas SciML model design requires reasoning across architectural choices, physical constraints, solver behavior, training dynamics, and numerical stability. Similarly, symbolic regression methods^{21,22} recover interpretable expressions but do not construct PDE-constrained learning architectures. Recent automated PINN and operator-learning frameworks³⁵ streamline training workflows but do not modify the underlying modeling paradigm. AgenticSciML differs in aiming for the *construction* of new solution strategies rather than the

optimization of predefined ones, with collaborative critique and retrieval enabling iterative refinement informed by prior results.

We also note some current limitations of this work. The quality of emergent solutions depends in part on the structure and coverage of the knowledge base, and improving retrieval grounding remains an important direction. Debate and justification among agents are mediated by LLMs and thus may not always reflect physically rigorous reasoning unless paired with numerical verification signals. Furthermore, the evolutionary search layer introduces computational overhead proportional to evaluation cost, suggesting that future work should prioritize tighter integration with differentiable solvers and low-fidelity model proxies. Extending the framework to multi-physics systems, turbulence, data assimilation, and real experimental workflows will also require adaptation in both evaluation procedures and memory representation.

Future directions include incorporating classic solvers, e.g., finite element and spectral methods, as well as hybrid methods based on PINNs or neural operators combined with classical methods. This will provide scalability and improved accuracy. In addition, we plan to develop stronger *physics-grounded reasoning signals*, such as adjoint-based consistency checks or solver-in-the-loop validation; exploring *hierarchical agent coordination* in which meta-agents learn to orchestrate debate and search strategies; and formalizing the relationship between collaborative agentic dynamics and the emergence of new SciML strategies. More broadly, our results suggest that multi-agent reasoning systems offer a path toward scalable exploration of the combinatorial solution spaces characteristic of modern scientific computing that includes both classical solvers and neural PDEs and operators.

Methods

Our evolutionary multi-agent system operates in three phases (Figure 4): user input, data analysis and evaluation criteria creation, and solution evolution. A team of 10+ agents is deployed with specialized roles. Key steps are detailed below.

Structured User Input

To start, the human user provides structured input written in Markdown and JSON formats to the agentic system: (1) `Problem.md`: a statement of the problem to solve; (2) `Requirements.md`: constraints on implementation (frameworks, libraries, hardware); (3) `Evaluation.md`: metrics and procedures for scoring solutions; (4) `Data_config.json` (optional): training or validation datasets with descriptions, file paths, and loading instructions.

The problem and requirement statements are to provide context. The evaluation criteria define what success looks like for this problem, such as a low mean squared error on a validation set or low PDE residuals. Datasets are optional, and the user can choose to provide none, some, or all data needed for training or validation. Agents will use user-provided data when available, but can also generate synthetic data if needed. The user provides minimal natural language prompts, and the agents will handle all technical details. See Supplementary Section S1 for examples.

Exploratory Data Analysis

If training data is provided, a multi-modal (text and vision) data analyst agent autonomously generates Python code to perform exploratory data analysis on the training dataset. It is prompted to quantitatively and visually examine both mathematical properties (discontinuities, sharp gradients, multi-scale phenomena, etc.) and data quality (outliers, distribution characteristics, etc.). The agent then analyzes the generated plots using its vision capabilities and produces a text-only report (`data_analysis.md`) summarizing key findings, so that downstream text-only agents can always reference this analysis.

This step is important because human scientists often start by visualizing and analyzing data to gain intuitions. The data analyst agent mimics this process and provides such data intuition to all downstream agents. Note that no analysis is performed on validation data to avoid data leakage.

Evaluation Contract Generation

Before proceeding to solution generation, an evaluator agent formalizes an evaluation contract that all candidate solutions must satisfy. The goal is to ensure consistent evaluation across diverse solutions and produces a scalar score for ranking. Note that throughout this work, the score is usually defined as a loss to minimize such as mean squared error (MSE) or relative L_2 error, so lower is better.

The evaluator agent generates two components that define the solution interface. First, `evaluate.py` is a complete test script that loads existing model checkpoints, generates or loads validation data, computes metrics, and returns a score. Second, `guidelines.md` outlines some implementation details for the engineer agent to follow, such as model

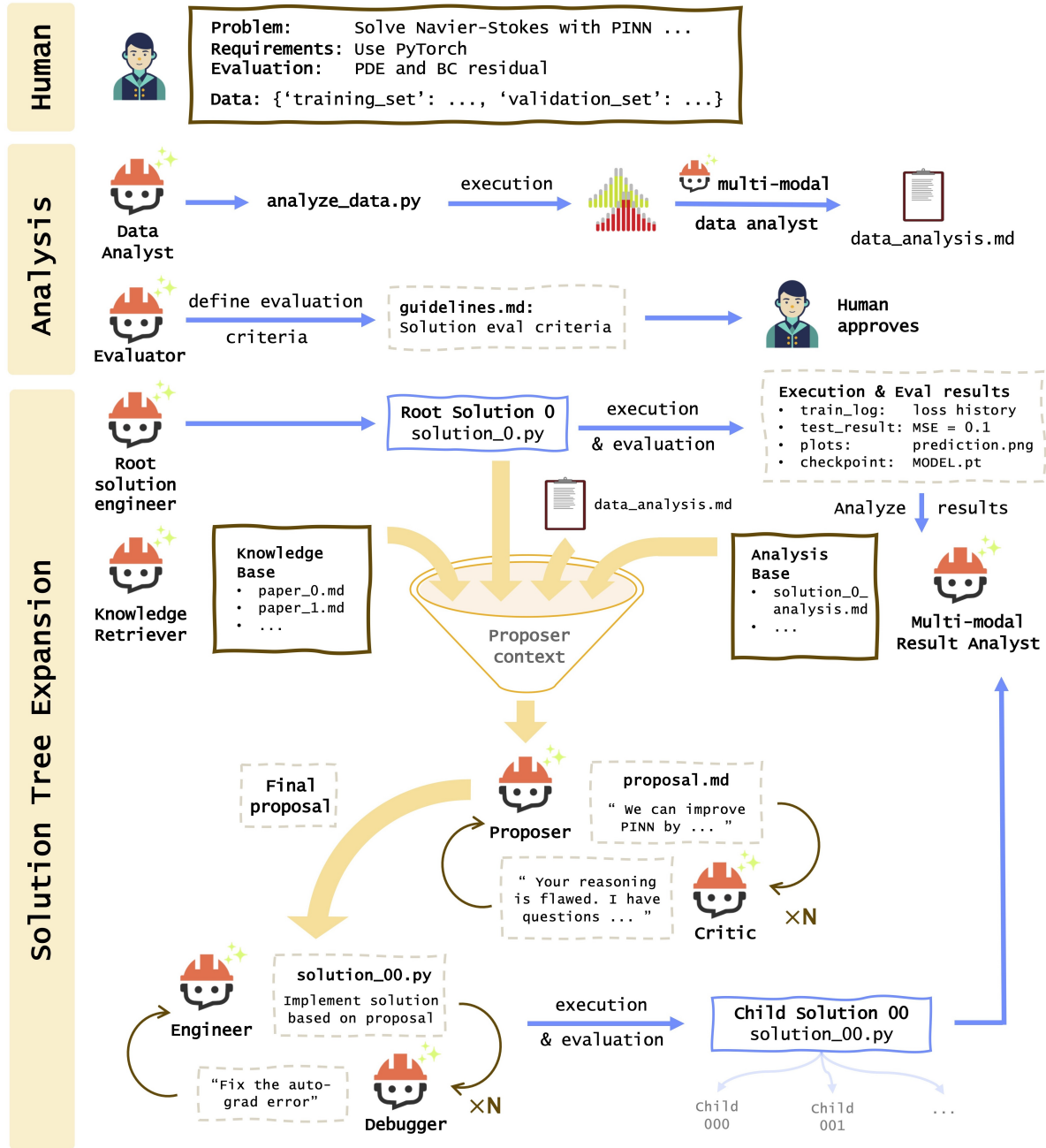


Figure 4

checkpoint formats and expected input-output data shapes, so that all engineered models can be evaluated through the same procedure. Finally, since the human defines the success criteria, human approval is sought before the plan is set.

Root Solution Generation

To start the solution tree expansion, a root solution is generated by a single-agent root engineer.

The root engineer has access to all the user inputs and evaluation guidelines. It produces a baseline solution, named `solution_0`. The solution is executed and evaluated autonomously. The root solution serves as our *single-agent baseline*: it is generated by a single LLM call with no access to the knowledge base, no multi-agent debate or critique,

and no prior solution context. This design choice isolates the contribution of multi-agent collaboration, knowledge retrieval, and evolutionary search from the raw capability of the underlying single-agent LLM. The results, including training logs and any generated plots, are analyzed by a multi-modal result analyst agent, which writes a detailed textual report (analysis_0.md) saved to the Analysis Base for future reference.

Ensemble-Guided Parent Selection

During each iteration of tree expansion, several existing solutions are selected from the solution tree to serve as parents for mutation. Each parent will produce one child solution through the mutation process described later. Multiple parents are mutated in parallel to generate multiple children per iteration.

In the early stage (total solutions $\leq$ mutation batch size), all solutions mutate in parallel. In the mature stage (total solutions $>$ mutation batch size), the best-scoring solution is always selected (exploitation), while additional parents are chosen through majority voting by a diverse ensemble of selector agents (exploration). Figure 5 shows an example of the selection mechanism.

The selectors are prompted to holistically evaluate solution code, analysis reports, and scores to select 1-2 solutions that give promising results and 1-2 solutions that are less promising but have potential for improvement. Each selector agent votes independently, and the solutions with the most votes are selected as additional parents. Solutions reaching maximum children per node (10) are excluded from selection to prevent over-exploitation.

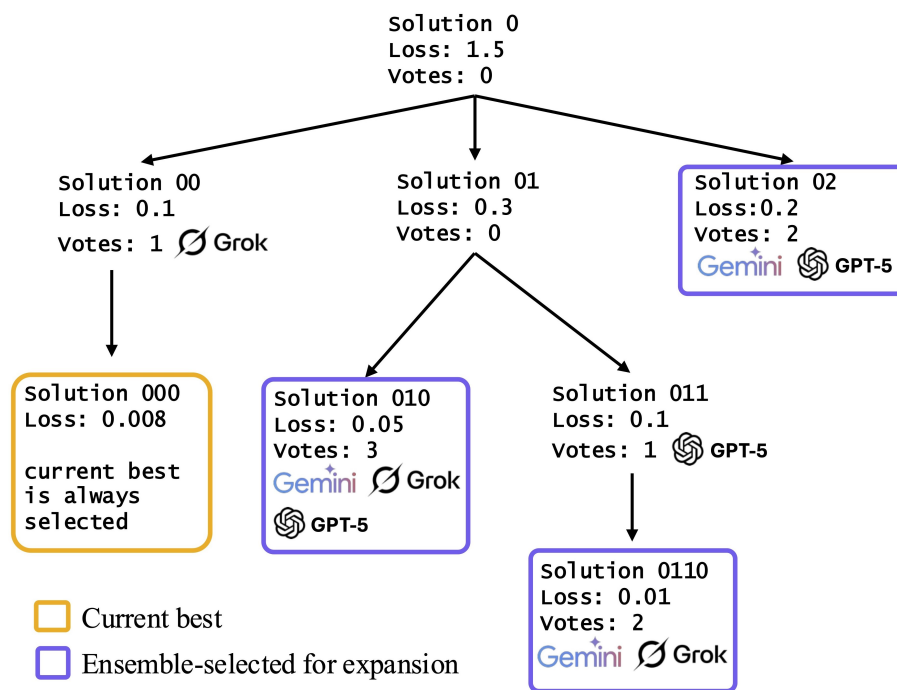


Figure 5

Solution Mutation through Multi-Agent Debate and Engineering

After parents are selected, child solutions are generated through three stages: planning, engineering, and evaluation.

Planning: This is the critical stage for scientific reasoning and innovation. First, a retriever agent critically evaluates the parent's solution and result analysis to determine if any entries from a curated Knowledge Base of SciML techniques would be helpful. If so, it retrieves 0-1 entries to share with the proposer and critic agents. More details on the Knowledge Base construction are provided in the Knowledge Base subsection below.

Next, several reports from the Analysis Base are also retrieved. These include the parent's analysis report, sibling reports (same parent), and uncle reports (grandparent's other children). This provides broader context on what strategies have worked or failed in neighboring solutions.

All these contexts are provided to the proposer and critic agents, who then engage in structured N -round debate to formulate a concrete implementation proposal for the child solution. For the first $N - 2$ rounds, the proposer is asked to analyze the problem deeply and think out loud. It is *not* asked to propose a new strategy yet – it focuses solely on reasoning. The critic then challenges the proposer’s reasoning, pointing out flaws, gaps, or alternative perspectives. In the $(N - 1)$ -th round, the proposer starts to synthesize its reasoning into a concrete implementation plan, which the critic evaluates for feasibility and soundness. Finally, in the N -th round, the proposer produces a final implementation-ready proposal without further critique.

Engineering: The engineer agent receives the parent’s full code, final proposal, and other problem specifications. It implements the exact proposed solution by modifying the parent’s codebase and does not need to reason about high-level strategies. Any errors are handled by the debugger agent until successful execution or maximum debugging iteration reached.

Evaluation: The multi-modal result analyst agent is again employed to analyze the child solution’s results, both textual logs and generated plots, and write a detailed report, which is added into the Analysis Base. The score of this solution is also computed for ranking.

Iteration

The above tree mutation process is repeated for a fixed number of iterations or until a stopping criterion is met. At the end, the best-scoring solution on the tree is returned as the champion solution.

Algorithm Summary

Algorithm 1 consolidates the three-phase procedure. The LLM assigned to each agent role varies by experiment and is summarized in the Supplementary Tables S1–S6. By default: Gemini 2.5 Pro/Flash serves as Proposer, Retriever, Data Analyst, and Result Analyst; Claude Haiku 4.5 as Root Engineer and Engineer; GPT-5 Mini as Critic and Debugger; and GPT-5 Mini, Grok-4 Fast, and Gemini as the Selector ensemble.

Knowledge Base

Construction of the Knowledge Base

The knowledge base (KB) used in our experiments consists of 70 entries of SciML techniques. While it is generally advantageous to include *only* the entries relevant to a specific problem context, we keep all entries in the KB throughout our experiments to test the retriever agent’s robustness, even though many entries are not directly relevant and can potentially lead to agents’ confusion.

Each KB entry is constructed by synthesizing information from both research papers and their corresponding publicly available code on GitHub. The curation process is semi-automated using an LLM agent (Claude). Human researchers first identify relevant papers and code, and the agent then extracts key information including:

- **Problem Setup:** The specific scientific computing problem being addressed, including equation types, boundary conditions, and domain characteristics.
- **Issues addressed:** Concrete symptoms and challenges the method mitigates (e.g., slow convergence, oscillations, discontinuities, noise sensitivity), so that agents can match methods to observed training difficulties.
- **Core method:** The key algorithmic or architectural innovation.
- **Implementation:** Annotated code snippets on the model architecture, training loop, and other algorithmic components.
- **Critical parameters:** Hyperparameters and design choices.

If a paper discussed multiple applications, all applications are included in the knowledge base as separate entries under the same method.

All entries are stored as structured Markdown files. A summary index in JSON format holds all the method names, descriptions, and file paths. The retriever agent will search through this index to identify relevant methods based on the problem context and training issues reported by other agents.

An example knowledge base entry is provided in Figure 6.

Algorithm 1 AgenticSciML: Evolutionary Multi-Agent SciML Framework**Require:** Problem P , Requirements R , Evaluation criteria E , optional Data D **Ensure:** Champion SciML solution s^* **Phase 1: Initialization**

```

1: if  $D$  is provided then
2:    $r_{\text{data}} \leftarrow \text{DATAANALYST}(D)$   $\triangleright$  Multimodal EDA; saves text-only report for downstream agents
3: end if
4:  $(\text{eval.py}, \text{guidelines.md}) \leftarrow \text{EVALUATOR}(P, R, E)$   $\triangleright$  Formalize testing contract
5: Await human approval of  $(\text{eval.py}, \text{guidelines.md})$   $\triangleright$  Human-in-the-loop checkpoint

```

Phase 2: Root Solution

```

6:  $s_0 \leftarrow \text{ROOTENGINEER}(P, R, \text{guidelines.md})$   $\triangleright$  Single-agent baseline: no KB, no debate
7:  $\text{score}_0 \leftarrow \text{EXECUTE}(s_0, \text{eval.py})$ 
8:  $a_0 \leftarrow \text{RESULTANALYST}(s_0, \text{score}_0)$   $\triangleright$  Multimodal analysis report
9: Initialize tree  $\mathcal{T} \leftarrow \{(s_0, \text{score}_0, a_0)\}$ , Analysis Base  $\mathcal{A} \leftarrow \{a_0\}$ 

```

Phase 3: Evolutionary Tree Expansion

```

10: for  $t = 1, 2, \dots, T_{\text{max}}$  do
11:    $\mathcal{P} \leftarrow \{\arg \min_{s \in \mathcal{T}} \text{score}(s)\}$   $\triangleright$  Always include best solution (exploitation)
12:    $\mathcal{P} \leftarrow \mathcal{P} \cup \text{SELECTORENSEMBLE}(\mathcal{T}, K-1)$   $\triangleright K-1$  additional parents by majority vote (exploration)
13:   for each parent  $p \in \mathcal{P}$  do  $\triangleright$  All parents mutated in parallel
14:      $\text{kb} \leftarrow \text{RETRIEVER}(p, \mathcal{T}, \text{KB})$   $\triangleright$  Retrieve 0–1 relevant KB entry based on parent’s weaknesses
15:      $\text{ctx} \leftarrow \mathcal{A}.\text{get}(p)$   $\triangleright$  Retrieve parent, sibling, and uncle analysis reports
16:     for round  $r = 1$  to  $N$  do  $\triangleright$  Structured  $N$ -round debate ( $N = 4$  in all experiments)
17:       if  $r \leq N - 2$  then  $\triangleright$  Reasoning rounds: deep analysis without proposing
18:          $\text{reasoning}_r \leftarrow \text{PROPOSER}(\text{ctx}, \text{kb}, \text{critique}_{r-1})$ 
19:          $\text{critique}_r \leftarrow \text{CRITIC}(\text{reasoning}_r)$   $\triangleright$  Challenge flaws, gaps, and alternatives
20:       else if  $r = N - 1$  then  $\triangleright$  Synthesis round: form concrete implementation plan
21:          $\text{plan} \leftarrow \text{PROPOSER.SYNTHESIZE}(\text{reasoning}_{1 \dots N-2}, \text{kb})$ 
22:          $\text{critique}_{N-1} \leftarrow \text{CRITIC.EVALUATE}(\text{plan})$   $\triangleright$  Assess feasibility and soundness
23:       else  $\triangleright r = N$ : finalization; no further critique
24:          $\text{proposal} \leftarrow \text{PROPOSER.FINALIZE}(\text{plan}, \text{critique}_{N-1})$ 
25:       end if
26:     end for
27:      $c \leftarrow \text{ENGINEER}(p.\text{code}, \text{proposal}, \text{guidelines.md})$   $\triangleright$  Implement child by modifying parent code
28:     repeat  $c \leftarrow \text{DEBUGGER}(c)$  until no errors or max retries reached
29:      $\text{score}_c \leftarrow \text{EXECUTE}(c, \text{eval.py})$ 
30:      $a_c \leftarrow \text{RESULTANALYST}(c, \text{score}_c, a_p, \text{kb})$   $\triangleright$  Multimodal analysis; added to  $\mathcal{A}$ 
31:      $\mathcal{T} \leftarrow \mathcal{T} \cup \{(c, \text{score}_c, a_c)\}$ 
32:   end for
33: end for
34: return  $s^* = \arg \min_{s \in \mathcal{T}} \text{score}(s)$   $\triangleright$  Champion solution

```

List of Knowledge Base Entries

Table 4 provides a comprehensive overview of all KB entries.

Table 4: Knowledge Base Entries: 70 Scientific Machine Learning Methods

#	Method	Description	Ref.
1	Adaptive Act.	Adaptive activation functions for Burgers equation	36
2	Local Adaptive Act.	Locally adaptive activation with slope recovery	41
3	DeepONet	Antiderivative operator learning	3;45
4	PI-DeepONet	Physics-informed antiderivative operator learning	44;45

Continued on next page

Table 4 – Continued from previous page

#	Method	Description	Ref.
5	PI-DeepONet	Diffusion-reaction systems	44;45
6	MoE-PINN	Burgers equation with mixture-of-experts	42
7	MoE-PINN	Poisson equation on L-shaped domain	42
8	Self-Adaptive PINN	Allen-Cahn equation	38
9	Self-Adaptive PINN	Burgers equation	38
10	Self-Adaptive PINN	Helmholtz equation	38
11	PINN	Burgers equation forward problem	1
12	PINN	Burgers equation inverse problem	1
13	PINN	Diffusion equation with source term	1
14	PINN	Simple ordinary differential equation	1
15	PINN	1D Poisson equation with Dirichlet BC	1
16	hp-VPINN	Advection-diffusion inverse problem	43
17	hp-VPINN	1D Poisson equation	43
18	hp-VPINN	2D Poisson equation	43
19	hp-VPINN	2D Poisson equation on L-shaped domain	43
20	gPINN	Diffusion-reaction equation	37
21	Delta-PINN	Complex geometries	46
22	gPINN	Allen-Cahn equation with adaptive refinement	37
23	gPINN	Burgers' equation	37
24	gPINN	Inverse diffusion-reaction system	37
25	gPINN	Function approximation	37
26	gPINN	1D Poisson equation	37
27	gPINN	2D Poisson equation	37
28	NSFnet	Incompressible Navier-Stokes equations	47
29	ev-NSFnet	Entropy-viscosity regularized NSFnet	47
30	hPINN	Inverse design with hard constraints	48
31	hPINN	Stokes flow topology optimization	48
32	DeepONet	2D advection equations	49;3
33	FNO	2D advection equations	49;2
34	DeepONet	1D Burgers' equation	49;3

Continued on next page

Table 4 – Continued from previous page

#	Method	Description	Ref.
35	FNO	1D Burgers' equation	49;2
36	DeepONet	2D Darcy flow with piecewise constant permeability	49;3
37	FNO	2D Darcy flow	49;2
38	DeepONet	2D Darcy flow on triangular domain with notch	49;3
39	FNO	Irregular geometries	49;2
40	POD-DeepONet	Instability wave propagation	49;3
41	FNO	Wave propagation with non-uniform grids	49;2
42	ICON	In-context operator networks	50
43	U-FNO	Gas saturation in CO ₂ storage	39
44	U-FNO	Pressure buildup in CO ₂ storage	39
45	CNO	Allen-Cahn equation	40
46	Fourier-DeepONet	Full waveform inversion (CurveFault-A)	51
47	Fourier-DeepONet	Full waveform inversion (CurveVel-A)	51
48	Fourier-DeepONet	Full waveform inversion (FlatVel-B)	51
49	Fourier-DeepONet	Full waveform inversion (Style-A)	51
50	CNO	Smooth transport equation	40
51	CNO	Darcy flow	40
52	CNO	Discontinuous transport equation	40
53	CNO	Compressible Euler equations (airfoil)	40
54	CNO	Navier-Stokes shear layer	40
55	CNO	Wave equation	40
56	CNO-FM	Foundation model for multiple PDEs	40
57	Oformer	Compressible flow around airfoils	52
58	Oformer	Burgers' equation	52
59	Oformer	Darcy flow	52
60	Oformer	Electrostatics Poisson equation	52
61	Oformer	Magnetostatics Poisson equation	52
62	Oformer	2D Navier-Stokes equations	52
63	GNOT	2D Darcy flow	53
64	GNOT	2D elasticity	53

Continued on next page

Table 4 – Continued from previous page

#	Method	Description	Ref.
65	GNOT	2D multi-scale heat conduction	53
66	GNOT	3D multi-physics heatsink	53
67	GNOT	2D electromagnetic inductor	53
68	GNOT	2D transonic airfoil flow	53
69	GNOT	2D Navier-Stokes steady-state	53
70	GNOT	2D time-dependent Navier-Stokes	53

Data Availability

Data generated and analyzed during this study are available in the code repository on GitHub: <https://github.com/Qile-J/AgenticSciML>. Knowledge base entries used in this work are derived from published papers and their associated publicly available code repositories. Additional information is available from the corresponding author upon request.

Code Availability

Code and experiment results are available on GitHub <https://github.com/Qile-J/AgenticSciML>. Additional information is available from the corresponding author upon request.

Acknowledgements

This work was supported by the Vannevar Bush Faculty Fellowship award from ONR (N00014-22-1-2795). The funder played no role in study design, data collection, analysis and interpretation of data, or the writing of this manuscript.

Author Contributions

G.K. and Q.J. conceived the research idea and experimental design. Q.J. implemented the framework and conducted all experiments. G.K. and Q.J. wrote and revised the manuscript.

Competing Interests

The authors declare no competing financial or non-financial interests.

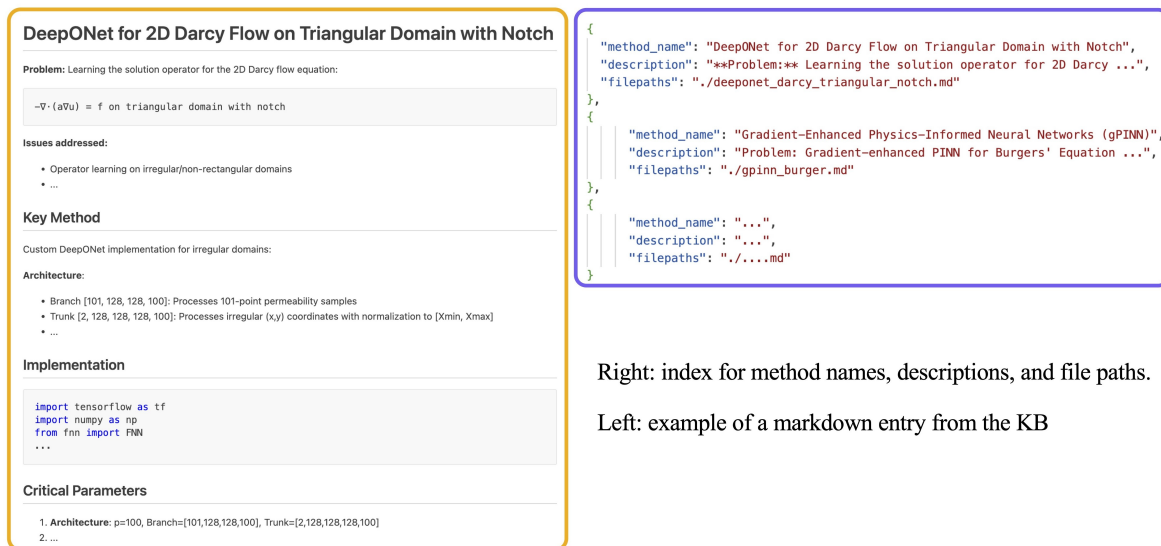


Figure 6

References

- [1] Raissi, M., Perdikaris, P. & Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019).
- [2] Li, Z. *et al.* Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895* (2020).
- [3] Lu, L., Jin, P., Pang, G., Zhang, Z. & Karniadakis, G. E. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nat. Mach. Intell.* **3**, 218–229 (2021).
- [4] Jagtap, A. D., Kharazmi, E. & Karniadakis, G. E. Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems. *Comput. Methods Appl. Mech. Eng.* **365**, 113028 (2020).
- [5] Meng, X., Li, Z., Zhang, D. & Karniadakis, G. E. Ppinn: Parareal physics-informed neural network for time-dependent pdes. *Comput. Methods Appl. Mech. Eng.* **370**, 113250 (2020).
- [6] Ren, S. *et al.* Towards scientific intelligence: A survey of llm-based scientific agents. *arXiv preprint arXiv:2503.24047* (2025).
- [7] Wei, J. *et al.* From ai for science to agentic science: A survey on autonomous scientific discovery. *arXiv preprint arXiv:2508.14111* (2025).
- [8] Swanson, K., Wu, W., Bulaong, N. L., Pak, J. E. & Zou, J. The virtual lab of ai agents designs new sars-cov-2 nanobodies. *Nature* **646**, 716–723 (2025).
- [9] Bran, A. M. *et al.* Chemcrow: Augmenting large-language models with chemistry tools. *arXiv preprint arXiv:2304.05376* (2023).
- [10] Lu, C. *et al.* The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292* (2024).
- [11] Gottweis, J. *et al.* Towards an ai co-scientist. *arXiv preprint arXiv:2502.18864* (2025).
- [12] Du, Y. *et al.* Accelerating scientific discovery with autonomous goal-evolving agents. *arXiv preprint arXiv:2512.21782* (2025).
- [13] Zhou, L. *et al.* Autonomous agents for scientific discovery: Orchestrating scientists, language, code, and physics. *arXiv preprint arXiv:2510.09901* (2025).
- [14] Georgiev, B., Gómez-Serrano, J., Tao, T. & Wagner, A. Z. Mathematical exploration and discovery at scale. *arXiv preprint arXiv:2511.02864* (2025).

- [15] Romera-Paredes, B. *et al.* Mathematical discoveries from program search with large language models. *Nature* **625**, 468–475 (2024).
- [16] Yu, Z. *et al.* Alpharesearch: Accelerating new algorithm discovery with language models. *arXiv preprint arXiv:2511.08522* (2025).
- [17] Shojaee, P., Meidani, K., Gupta, S., Farimani, A. B. & Reddy, C. K. Llm-sr: Scientific equation discovery via programming with large language models. *arXiv preprint arXiv:2404.18400* (2024).
- [18] Ou, Y. *et al.* Automind: Adaptive knowledgeable agent for automated data science. *arXiv preprint arXiv:2506.10974* (2025).
- [19] Jiang, Z. *et al.* Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138* (2025).
- [20] Press, O. *et al.* Algotune: Can language models speed up general-purpose numerical programs? *arXiv preprint arXiv:2507.15887* (2025).
- [21] Cranmer, M. *et al.* Discovering symbolic models from deep learning with inductive biases. *Adv. Neural Inf. Process. Syst.* **33**, 17429–17442 (2020). URL <https://arxiv.org/abs/2006.11287>.
- [22] Udrescu, S.-M. & Tegmark, M. Ai feynman 2.0: Pareto-optimal symbolic regression driven by iterative learning. *arXiv preprint arXiv:2006.10782* (2020). URL <https://arxiv.org/abs/2006.10782>.
- [23] Aygün, E. *et al.* An ai system to help scientists write expert-level empirical software. *arXiv preprint arXiv:2509.06503* (2025).
- [24] Ma, P. *et al.* Llm and simulation as bilevel optimizers: A new paradigm to advance physical scientific discovery. *arXiv preprint arXiv:2405.09783* (2024).
- [25] Cornelio, C., Ito, T., Cory-Wright, R., Dash, S. & Horesh, L. The need for verification in ai-driven scientific discovery. *arXiv preprint arXiv:2509.01398* (2025).
- [26] Miao, J., Davis, J. R., Zhang, Y., Pritchard, J. K. & Zou, J. Paper2agent: Reimagining research papers as interactive and reliable ai agents. *arXiv preprint arXiv:2509.06917* (2025).
- [27] Novikov, A. *et al.* Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131* (2025).
- [28] Lange, R. T., Imajuku, Y. & Cetin, E. Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349* (2025).
- [29] Liu, H., Simonyan, K. & Yang, Y. Darts: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)* (2019). URL <https://arxiv.org/abs/1806.09055>.
- [30] Real, E., Aggarwal, A., Huang, Y. & Le, Q. V. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 4780–4789 (2019).
- [31] Liu, Y. *et al.* Alphago moment for model architecture discovery. *arXiv preprint arXiv:2507.18074* (2025).
- [32] Wuwu, Q. *et al.* Pinnsagent: Automated pde surrogation with large language models. *arXiv preprint arXiv:2501.12053* (2025).
- [33] He, X. *et al.* Lang-pinn: From language to physics-informed neural networks via a multi-agent framework. *arXiv preprint arXiv:2510.05158* (2025).
- [34] Choi, H. K., Zhu, X. & Li, S. Debate or vote: Which yields better decisions in multi-agent large language models? *arXiv preprint arXiv:2508.17536* (2025).
- [35] Zhang, Y. *et al.* Auto-pinn: Automated physics-informed neural network training via algorithmic differentiation and meta-learning. *arXiv preprint arXiv:2303.05552* (2023). URL <https://arxiv.org/abs/2303.05552>.
- [36] Jagtap, A. D., Kawaguchi, K. & Karniadakis, G. E. Adaptive activation functions accelerate convergence in deep and physics-informed neural networks. *J. Comput. Phys.* **404**, 109136 (2020).
- [37] Yu, J., Lu, L., Meng, X. & Karniadakis, G. E. Gradient-enhanced physics-informed neural networks for forward and inverse pde problems. *Comput. Methods Appl. Mech. Eng.* **393**, 114823 (2022).
- [38] McClenny, L. D. & Braga-Neto, U. M. Self-adaptive physics-informed neural networks. *J. Comput. Phys.* **474**, 111722 (2023).
- [39] Wen, G., Li, Z., Azizadenesheli, K., Anandkumar, A. & Benson, S. M. U-fno—an enhanced fourier neural operator-based deep-learning model for multiphase flow. *Adv. Water Resour.* **163**, 104180 (2022).
- [40] Raonic, B., Molinaro, R., Rohner, T., Mishra, S. & de Bezenac, E. Convolutional neural operators. In *ICLR 2023 workshop on physics for machine learning* (2023).

- [41] Jagtap, A. D., Kawaguchi, K. & Em Karniadakis, G. Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks. *Proc. R. Soc. A* **476**, 20200334 (2020).
- [42] Bischof, R. & Kraus, M. A. Mixture-of-experts-ensemble meta-learning for physics-informed neural networks. In *Proceedings of 33. forum bauintformatik* (2022).
- [43] Kharazmi, E., Zhang, Z. & Karniadakis, G. E. hp-vpinns: Variational physics-informed neural networks with domain decomposition. *Comput. Methods Appl. Mech. Eng.* **374**, 113547 (2021).
- [44] Wang, S., Wang, H. & Perdikaris, P. Learning the solution operator of parametric partial differential equations with physics-informed deepnets. *Sci. Adv.* **7**, eabi8605 (2021).
- [45] Toscano, J. D. Learning-scientific machine learning: Residual based attention pinns, pikans, and deepnets. https://github.com/jdtoscano94/Learning-Scientific_Machine_Learning_Residual_Based_Attention_PINNs_PIKANs_DeepONets (2024).
- [46] Costabal, F. S., Pezzuto, S. & Perdikaris, P. δ -pinns: Physics-informed neural networks on complex geometries. *Eng. Appl. Artif. Intell.* **127**, 107324 (2024).
- [47] Jin, X., Cai, S., Li, H. & Karniadakis, G. E. Nsfnets (navier-stokes flow nets): Physics-informed neural networks for the incompressible navier-stokes equations. *J. Comput. Phys.* **426**, 109951 (2021).
- [48] Lu, L. *et al.* Physics-informed neural networks with hard constraints for inverse design. *SIAM J. Sci. Comput.* **43**, B1105–B1132 (2021).
- [49] Lu, L. *et al.* A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data. *Comput. Methods Appl. Mech. Eng.* **393**, 114778 (2022).
- [50] Yang, L., Liu, S., Meng, T. & Osher, S. J. In-context operator learning with data prompts for differential equation problems. *Proc. Natl Acad. Sci.* **120**, e2310142120 (2023).
- [51] Zhu, M., Feng, S., Lin, Y. & Lu, L. Fourier-deeponet: Fourier-enhanced deep operator networks for full waveform inversion with improved accuracy, generalizability, and robustness. *Comput. Methods Appl. Mech. Eng.* **416**, 116300 (2023).
- [52] Li, Z., Meidani, K. & Farimani, A. B. Transformer for partial differential equations' operator learning. *arXiv preprint arXiv:2205.13671* (2022).
- [53] Hao, Z. *et al.* Gnot: A general neural operator transformer for operator learning. In *International Conference on Machine Learning*, 12556–12569 (PMLR, 2023).

Figure Legends

Figure 1. Multi-agent system performance improvements over single-agent baselines across six benchmark SciML problems. Each bar shows the improvement factor achieved by the multi-agent evolutionary framework over the single-agent root solution for one of the six benchmark problems: discontinuous function approximation (Func. Approx.), Poisson equation on an L-shaped domain (Poisson), Burgers' equation using a physics-informed neural network (Burger's PINN), antiderivative operator learning (Antiderivative), multiple-input operator learning for a reaction-diffusion equation (Multi-input Op.), and 2D cylinder wake vorticity reconstruction (Cylinder Wake). The improvement factor is defined as the ratio of the root solution score to the champion solution score, where the score is a loss metric (lower is better); a higher improvement factor indicates a greater benefit from multi-agent collaboration, knowledge retrieval, and evolutionary search. The y-axis is displayed on a base-10 logarithmic scale. In all six experiments the multi-agent system achieves a lower score than the single-agent baseline, with improvement factors ranging from approximately 10 to over 11,000.

Figure 2. Agreement among selector agents for the top three nominations across all experiments. Each bar shows the fraction of experiments in which all three selector models — GPT-5 Mini, Grok-4 Fast, and Gemini 2.5 Pro/Flash — cast identical votes for a given nomination rank. The x-axis shows the nomination rank (1st choice, 2nd choice, 3rd choice) and the y-axis shows the fraction of experiments (out of six total) in which the three selectors were unanimous. High agreement at the 1st-rank nomination reflects consistent identification of the best-performing candidate for exploitation. Lower agreement at the 3rd-rank nomination reflects healthy divergence in exploratory candidate selection, where selectors disagree on which lower-performing solution has the most potential for further improvement. The three bars correspond to the first, second, and third nomination ranks respectively.

Figure 3. Agent word-count contributions across all experiments. The stacked bar chart shows the total word count contributed by each planning agent type in each of the six experiments. The x-axis labels each experiment (Func. Approx., Poisson, Burger's PINN, Antiderivative, Multi-input Op., Cylinder Wake) and the y-axis shows total word count. Each colored stacked segment represents one agent type: Proposer, Critic, User, Retriever, and Result Analyst; the color assigned to each agent type is indicated in the figure legend. Engineering and selector agents are excluded. The Proposer agent contributes the largest share of words in all experiments, owing to its multi-round structured reasoning steps prior to and during proposal synthesis. The human user contributes less than 0.3% of the total generated text across all six experiments, demonstrating the high degree of autonomy of the multi-agent system.

Figure 4. Overview of the AgenticSciML evolutionary multi-agent framework. The diagram illustrates the three-phase structure of the framework. In Phase 1, the human user provides four structured inputs: a problem statement (Problem.md), implementation requirements (Requirements.md), evaluation criteria (Evaluation.md), and an optional dataset configuration (Data_config.json). In Phase 2, a Data Analyst agent performs exploratory data analysis on any provided training data and produces a text report for downstream agents; an Evaluator agent formalizes a testing contract (evaluate.py and guidelines.md) that is approved by the human user before proceeding. In Phase 3, the evolutionary loop runs iteratively: a Retriever agent selects relevant entries from the curated Knowledge Base (KB); Proposer and Critic agents engage in structured multi-round debate to produce an implementation proposal; an Engineer agent implements the proposal by modifying the parent solution's code; a Debugger agent resolves runtime errors; and a Result Analyst agent evaluates the solution output and writes a report added to the Analysis Base. The process repeats until a stopping criterion is met, at which point the best-scoring solution on the tree is returned as the champion solution.

Figure 5. Illustration of the ensemble-guided solution tree selection mechanism. The diagram shows a representative solution tree at one evolutionary iteration. Each node represents a candidate solution annotated with its score. The best-performing node currently on the tree (highlighted with a star) is always selected for mutation in the next iteration to ensure exploitation of the strongest known solution. One or more additional nodes are selected by majority vote among the selector ensemble (GPT-5 Mini, Grok-4 Fast, Gemini 2.5 Pro/Flash) to ensure exploration of alternative directions. Arrows connecting nodes indicate parent-child relationships produced by mutation. All selected parent nodes are mutated in parallel, each producing one child solution per iteration. Nodes that have reached the maximum number of children (10) are excluded from selection to prevent over-exploitation of a single branch.

Figure 6. Example knowledge base entry and the JSON index summarizing method descriptions and file paths. Each KB entry is stored as a structured Markdown file containing the problem setup, issues addressed, core method, implementation code snippets, and critical hyperparameters. The JSON index holds a summary of all 70 entries (method name, brief description, and file path) and is used by the retriever agent to identify the most relevant KB entry for a given parent solution.